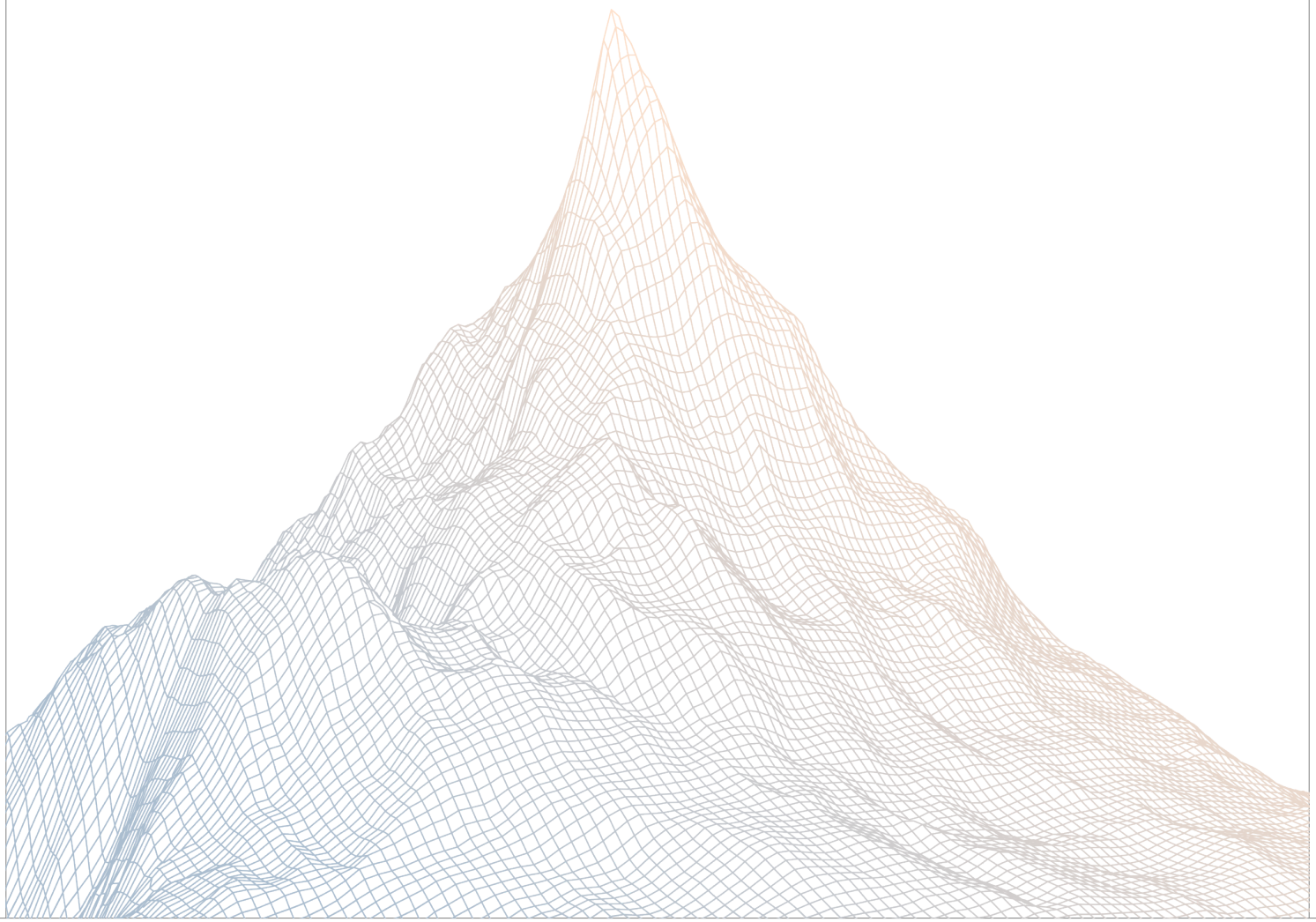
Zenith

c8ntinuum

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: April 11, 2026

AUDITED BY: adriro
said

Contents

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About c8ntinuum

c8ntinuum is an aggregated multi-chain interoperability protocol that employs a trustless and permissionless infrastructure layer for transferring information and value between connected chains.

2.2 Scope

The engagement involved a review of the following targets:

Target	0xc8fb80fcc03f699c70ff0cc08c09106288888888
Repository	https://etherscan.io/token/0xc8...888
Target	0xc8Fb80fCc03f699C70ff0CC08C09106288888888
Repository	https://bscscan.com/address/0xc8...88

2.3 Audit Timeline

April 11, 2026	Audit start
April 11, 2026	Audit end
April 15, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0
Informational	4
Total Issues	**4**

3

Findings Summary

ID	Description	Status
I-1	MAX_SUPPLY check does not enforce a global supply invariant with burn-and-mint bridging	Acknowledged
I-2	Immutable DEFAULT_ADMIN address does not prevent role mutations	Acknowledged
I-3	Code style improvements	Acknowledged
I-4	Missing burnFrom function may block mint/burn bridge integration	Acknowledged

4

Findings

4.1 Informational

A total of 4 informational findings were identified.

[I-1] MAX_SUPPLY check does not enforce a global supply invariant with burn-and-mint bridging

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- ctm.sol

Description:

The `mint()` function enforces `totalSupply() + value ≤ MAX_SUPPLY` on a per-contract basis. Under a burn-and-mint bridging mechanism, tokens are burned on the source chain and minted on the destination chain. Because each chain's CTM deployment tracks its own `totalSupply()` independently, the `MAX_SUPPLY` check only limits supply on that specific chain rather than across all deployments.

In practice this means the aggregate circulating supply across all chains could exceed `MAX_SUPPLY` without any single chain violating its local check. The severity is low because legitimate bridge operations should preserve total supply (burn equals mint), but the invariant as written does not reflect a true global cap.

Recommendations:

If a global supply cap is intended, consider coordinating supply accounting across chains via an oracle or bridge-level enforcement, or document that `MAX_SUPPLY` is a per-chain limit by design.

Continuum: Acknowledged. The MAX_SUPPLY check is implemented as an additional defense mechanism. The global supply is managed 1:1 via our native blockchain's locking bridge and intermediary network of bridge operators which are connected to all networks. We have intentionally included this per-chain cap of 8.8 B as a safety ceiling.

[I-2] Immutable `DEFAULT_ADMIN` address does not prevent role mutations

`SEVERITY:` Informational	`IMPACT:` Informational
`STATUS:` Acknowledged	`LIKELIHOOD:` Low

Target

- ctm.sol

Description:

The `DEFAULT_ADMIN` is declared as a `constant` and hardcoded to a Safe multisig wallet, which gives a strong impression of fixed, transparent governance. However, `AccessControl` allows any account with the `DEFAULT_ADMIN_ROLE` to call `grantRole()` and `revokeRole()`, meaning the admin role itself — and the `MINTER_ROLE` — can be transferred to arbitrary addresses at any time.

This creates a gap between the perceived immutability conveyed by the `constant` keyword and the actual mutability of the role system. Users or integrators inspecting the contract may incorrectly assume that only the hardcoded address can ever hold admin or minter privileges.

Recommendations:

Consider documenting this behavior prominently or overriding `grantRole()` / `revokeRole()` to restrict which roles can be reassigned, if tighter guarantees are intended.

Continuum: Acknowledged We disagree with the characterization that the implementation is "misleading" or creates a "gap in perceived immutability". The use of a constant address for the DEFAULT_ADMIN is intended to provide a transparent starting point for the contract's governance.

[I-3] Code style improvements

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- ctm.sol

Description:

Several code style issues were identified in the contract:

Unaliased imports. The contract uses unaliased imports (e.g., `import "@openzeppelin/ ... /AccessControl.sol"`) which bring all symbols from the imported file into scope. Named imports (e.g., `import {AccessControl} from "..."`) are preferred as they make dependencies explicit and reduce the risk of naming collisions.

Mixed usage of *`uint`* and *`uint256`*. The contract uses `uint` in some places (`MAX_SUPPLY`, `burn()`) and `uint256` in others (`mint()`). While `uint` is an alias for `uint256`, mixing both reduces readability and consistency.

Revert strings instead of custom errors. The `mint()` function uses a `require` statement with a string literal for error handling. Since Solidity 0.8.26, custom errors are available and are more gas-efficient, as they avoid storing and emitting string data on-chain.

Recommendations:

Use named imports for all dependencies, standardize on `uint256` across the contract, and replace `require` with revert strings with custom errors.

Continuum: Acknowledged. I will adopt named imports, consistent uint256 usage, and custom errors in future versions of our contracts.

[I-4] Missing `burnFrom` function may block mint/burn bridge integration

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- CTM.sol

Description:

The CTM contract provides a `burn(uint value)` function that only burns tokens from `msg.sender`'s balance. There is no `burnFrom(address account, uint256 value)` function that would allow an approved spender (such as a bridge contract) to burn tokens directly from a user's wallet using an allowance.

If the minter role will be assigned to a bridge contract using a mint/burn mechanism. In a standard mint/burn bridge flow:

1. User approves the bridge contract to spend their CTM tokens
2. Bridge burns tokens on the source chain
3. Bridge mints (or unlocks) tokens on the destination chain

Without `burnFrom`, the bridge must use a two-step workaround:

1. `transferFrom(user, bridge, amount)` — move tokens to the bridge's own balance
2. `burn(amount)` — bridge burns from its own balance

This pattern consumes more gas than a single `burnFrom` call.

If the bridge implementation expects a standard `burnFrom` interface (as many bridge SDKs do), integration will require a custom adapter.

Recommendations:

Add a `burnFrom` function that consumes the caller's allowance before burning.

Continuum: Acknowledged. We agree that `burnFrom` is a standard optimization for bridge integrations, we have evaluated the trade-offs and decided not to implement it. The current two-step process meets our functional requirements, and we are comfortable accepting the slightly higher gas costs in exchange for keeping the token contract's logic minimal.